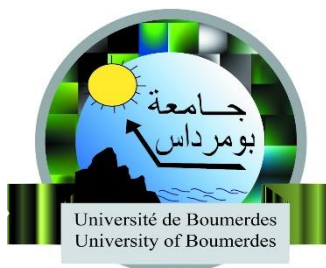


République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université M'hamed Bougara- Boumerdès



Faculté des Sciences
Département d'Informatique
Domaine : Mathématiques et Informatique
Filière : Informatique Spécialité : I2A

Projets- WEB AVANCÉ MICROSERVICES(WAMS)

Thème : Système E-commerce Intelligent de recherche et classement visuel

Présenté par :

HAICHEUR Said
GHERNOUTI Chihab Eddine
HAMDY Imad
MAHAMMED OUSAID Hichem
SLIMANE Kader Nabil

Groupe : 2

Résumé

Ce rapport décrit le projet Apex Motors de façon détaillée. Le système est organisé en plusieurs services Django séparés, chacun ayant un rôle précis : authentification, catalogue, commandes, recherche visuelle et passerelle principale. L'objectif est de maintenir un code lisible, de faciliter le déploiement local et de démontrer une vraie séparation entre la logique métier et la partie intelligence artificielle.

Le projet ne se limite pas à un CRUD classique. Il ajoute une recherche par image avec embeddings et Qdrant, des règles de permission basées sur le rôle utilisateur, des commandes avec snapshots, et une orchestration complète avec Docker Compose et Nginx.

Table des Matières

Introduction

1. Étude de l'existant
 - 1.1. Problématique
 - 1.2. Objectif
2. Analyse des besoins
3. Identification des acteurs
4. Architecture Globale de la Plateforme
 - 4.1. Composants de l'Architecture
5. Description des Microservices
 - 5.1. Service d'Authentification
 - 5.2. Service Catalogue
 - 5.3. Service commandes
 - 5.4. Service recherche
6. Communication entre APIs
7. Sécurité et permissions
8. Déploiement Docker
9. Présentation des Interfaces
 - 9.1. Page d'authentification
 - 9.2. Formulaire d'inscription
 - 9.3. Page Vendeur
 - 9.4. Page Client

Conclusion

Introduction

Apex Motors est une application e-commerce de démonstration centrée sur les voitures et les produits liés à l'automobile. Le projet illustre comment construire une plateforme plus réaliste qu'un simple monolithe. L'application est découpée en plusieurs services indépendants pour éviter que tout le code soit concentré dans un seul endroit.

Cette approche a un intérêt pédagogique fort. On voit comment séparer les responsabilités, comment faire communiquer plusieurs API REST, comment gérer des bases de données distinctes, et comment intégrer une brique de recherche intelligente sans alourdir le reste du système.

Dans les documents de référence du dépôt, le même esprit apparaît : architecture distribuée, services nommés clairement, proxy public, bases séparées et parcours utilisateurs bien définis.

1. Étude de l'existant

1.1. Problématique

Actuellement, pour acheter ou vendre un véhicule en Algérie, l'utilisateur se retrouve face à des solutions fragmentées et peu fiables : se déplacer physiquement dans un marché désigné, parcourir manuellement Facebook Marketplace sans aucune garantie de qualité, ou chercher sur Ouedkniss avec des filtres très basiques. Dans tous ces cas, il n'existe aucun moyen de rechercher un véhicule par image, aucune gestion des rôles entre acheteur et vendeur, et aucune traçabilité des transactions.

1.2. Objectif

Notre objectif est de créer une plateforme e-commerce qui facilite la recherche et l'achat de véhicules pour l'utilisateur algérien. Au lieu de chercher manuellement dans des listes interminables, le client peut simplement uploader une photo d'un véhicule qui l'intéresse, et le système lui propose automatiquement les annonces les plus similaires.

Pour réaliser cela, nous avons construit Apex Motors comme un ensemble de services indépendants : chaque partie du système (authentification, catalogue, commandes, recherche) fonctionne séparément et communique avec les autres via des APIs. Ce choix n'est pas uniquement technique, il permet au système de rester stable, facile à maintenir, et prêt à évoluer sans tout reconstruire à chaque changement.

2. Analyse des besoins

Les objectifs principaux du projet résident dans la mise en œuvre d'une architecture robuste capable de soutenir des flux hétérogènes. Le périmètre fonctionnel est rigoureusement segmenté selon les piliers suivants :

- Gestion identitaire et contrôle d'accès : Centralisation des profils et sécurisation des échanges.

- Administration du catalogue produit : Persistance des métadonnées automobiles et gestion des actifs multimédias.
- Orchestration transactionnelle : Gestion du cycle de vie des commandes avec garantie d'intégrité historique.
- Moteur de recherche par vision par ordinateur : Identification sémantique et vectorielle de produits à partir d'entrées visuelles.

3. Identification des acteurs

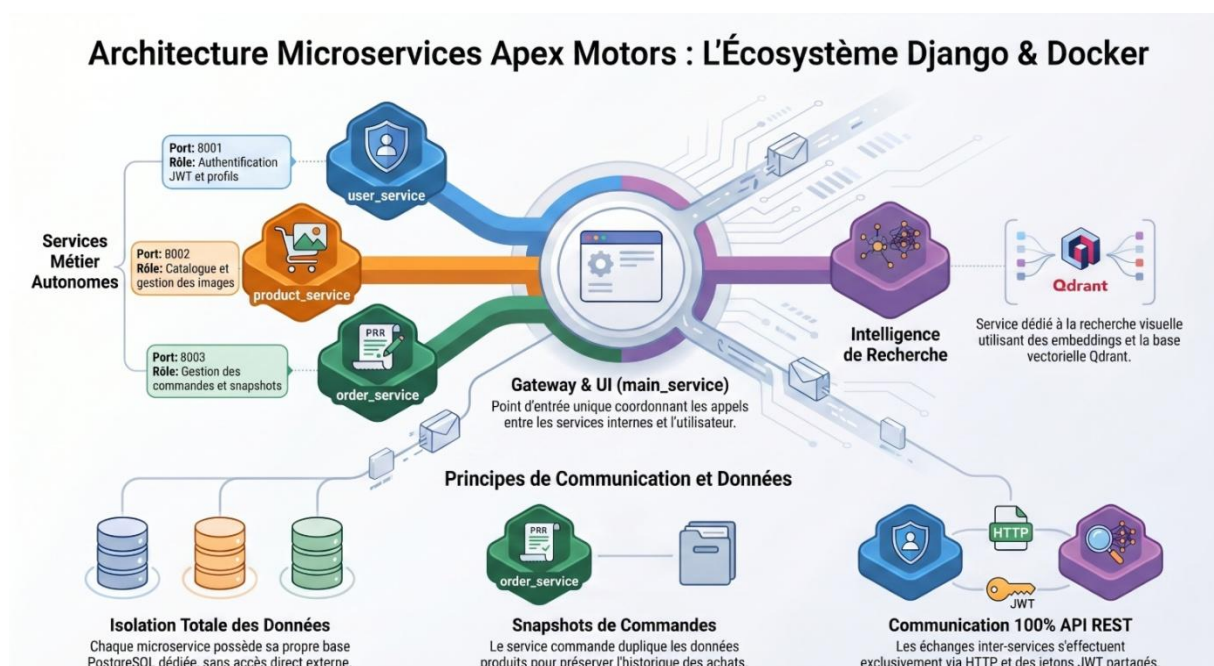
La plateforme cible trois types d'utilisateurs avec des rôles et responsabilités distincts :

User	Description détaillée
Admin	Ce type d'utilisateur ayant tout type accès et un Contrôle totale sur la plateforme comme gestion des produits, Ordre et gestion des utilisateur
Vendeur	Ayant le droit de crée un produit (Nom de produit, description détails de état de tell produit et un minimum de 5 images sous format JPEG, JPG, PNG) , et Modifier les produit
Client	Ayant le droit d'accès a la recherche des produit par image (Image retrieval) ,et faire des achat

4. Architecture Globale de la Plateforme

La plateforme repose sur une architecture microservices complète et découplée. Chaque service possède sa propre base de données PostgreSQL dédiée, garantissant l'isolation des données, la scalabilité indépendante de chaque composant, et la tolérance aux pannes. Cette approche « Database per Service » est un pattern fondamental des architectures microservices modernes.

Le choix de plusieurs bases séparées est important. Il évite les dépendances directes entre les tables des différents domaines. Par exemple, order_service ne lit pas directement les tables du catalogue — il interroge product_service via HTTP, puis stocke ses propres snapshots.



4.1. Composants de l'Architecture

Le système utilise cinq services applicatifs principaux : main_service, user_service, product_service, order_service et search_service. Chaque service a sa propre base PostgreSQL. En plus, Qdrant stocke les vecteurs d'images et Nginx sert de point d'entrée public.

Service	Port	Base de Données	Rôle Architectural
main_service	8000	postgres_main	Gateway Django et orchestration de l'interface utilisateur.
user_service	8001	postgres_users	Gestion de l'authentification stateless via SimpleJWT.
product_service	8002	postgres_products	Référentiel du catalogue et gestion des actifs médias.
order_service	8003	postgres_orders	Service transactionnel et archivage des snapshots.
search_service	8004	postgres_search	Pipeline IA et interfaçage avec la base vectorielle Qdrant.
qdrant	6333	-	Base de données vectorielle pour la recherche de similarité.
nginx	8080	-	Reverse proxy public agissant comme point d'entrée unique.

5. Description des Microservices

5.1. Service d'Authentification

user_service est le service responsable des comptes. Son rôle est simple mais central : créer les utilisateurs, gérer les profils et produire des tokens JWT. Le modèle UserProfile ajoute un champ role avec trois valeurs : admin, seller et client.

Le token JWT est enrichi avec le rôle et le nom d'utilisateur. C'est un choix pratique, car les autres services peuvent vérifier les droits sans appeler user_service à chaque requête. On garde ainsi une authentification stateless, plus adaptée à un système distribué

Route	Méthode	Rôle
/api/auth/register/	POST	Créer un compte
/api/auth/login/	POST	Obtenir un token
/api/auth/profile/	GET	Lire le profil

5.2. Service Catalogue

product_service est le cœur métier du catalogue. Il gère les catégories, les produits et les images associées. Le modèle Product contient le nom, la description, le prix, la catégorie, le vendeur et la date de création. Le modèle ProductImage conserve l'image, une miniature base64, un indicateur d'image principale, l'état d'indexation et l'identifiant Qdrant.

Un détail important est la génération de miniature dans le modèle. Le fichier image est ouvert, corrigé si nécessaire, redimensionné, compressé et converti en base64. Cela permet d'avoir un aperçu rapide sans charger tout le fichier dans l'interface.

Le service applique aussi des permissions métier. Un seller peut écrire dans le catalogue, mais un client ne le peut pas. De plus, un produit ne peut être modifié que par son propriétaire vendeur.

```

class ProductSerializer(serializers.ModelSerializer):
    category = CategorySerializer(read_only=True)
    category_id = serializers.PrimaryKeyRelatedField(
        source="category", queryset=Category.objects.all(), write_only=True, required=False, allow_null=True
    )
    images = ProductImageSerializer(many=True, read_only=True)
    seller_username = serializers.CharField(read_only=True)
    image_files = serializers.ListField(
        child=serializers.ImageField(),
        write_only=True,
        required=False,
        allow_empty=True,
    )

class Meta:
    model = Product
    fields = (
        "id",
        "name",
        "description",
        "price",
        "category",
        "category_id",
        "created_at",
        "seller_username",
        "images",
        "image_files",
    )

def validate(self, attrs):
    attrs = super().validate(attrs)
    image_files = attrs.get("image_files") or []
    if self.instance is None and len(image_files) < 5:
        raise serializers.ValidationError({"image_files": "Provide at least 5 images when creating a product."})
    return attrs

```

Cette validation montre une règle métier claire : la création d'un produit requiert au moins cinq images, alimentant ainsi la richesse de la recherche visuelle.

5.3. Service commandes

order_service gère les commandes de manière indépendante. Il enregistre l'utilisateur, la date, puis les lignes de commande avec des snapshots complets du produit. Cette méthode protège l'historique si le produit est modifié ou supprimé plus tard.

Le service utilise une transaction atomique. Cela veut dire que la commande entière est créée ou bien elle échoue complètement. C'est un point essentiel pour éviter les commandes partiellement enregistrées.

Au moment de la création, le service récupère les produits nécessaires depuis product_service. Il ne copie pas seulement l'identifiant produit, mais aussi le nom, la description, la catégorie et le prix unitaire. Le résultat est plus stable et plus lisible pour l'utilisateur final.

Route	Méthode	Rôle
/api/orders/	GET / POST	Lister ou créer une commande
/api/orders/<id>/	GET	Lire une commande précise

5.4. Service recherche

search_service est la partie la plus technique du projet. Il reçoit des images, les vérifie, les prépare, calcule des embeddings et interroge Qdrant pour retrouver des produits similaires. C'est la brique qui donne au projet son aspect intelligent.

Le pipeline est découpé en étapes simples : contrôle qualité de l'image, preprocessing, embedding, recherche vectorielle, filtrage et réponse finale. Cette découpe rend le code plus facile à suivre et plus facile à modifier si le modèle change.

Le service ne renvoie pas les fiches produits complètes. Il renvoie surtout des identifiants, des scores et des métadonnées. Ensuite, main_service récupère les détails du catalogue et reconstruit la réponse finale pour l'interface.

Route	Méthode	Rôle
/api/index/	POST	Indexer une image produit
/api/search/	POST	Rechercher une image similaire
/api/index/<product_id>/	DELETE	Supprimer les vecteurs d'un produit

```
def search_image(image_bytes, limit=10, score_threshold=None, model_id=None):
    ok, reason = validate_image_quality(image_bytes)
    if not ok:
        return {"detail": reason}
    vector = get_embedder(model_id).embed(preprocess_image_bytes(image_bytes))
    results = search_vectors(
        vector=vector, limit=limit,
        score_threshold=score_threshold, model_id=model_id)
    return {"matches": results}
```

6. Communication entre APIs

La communication entre services se fait presque toujours par HTTP et APIs REST. Le projet évite de partager directement des modèles Django entre services — chaque domaine reste indépendant. main_service joue le rôle de passerelle : le navigateur lui parle, puis il contacte les autres services selon l'action demandée.

Flux	Source	Cible	Résultat
Authentification	Navigateur	user_service	JWT avec rôle
Recherche image	main_service	search_service + product_service	Résultats hydratés
Ajout d'image	product_service	search_service + Qdrant	Indexation vectorielle
Commande	order_service	product_service	Snapshot de commande

7. Sécurité et permissions

Le projet applique une sécurité simple mais utile. Les rôles sont transmis dans le JWT. Les services utilisent ensuite ce rôle pour autoriser ou refuser certaines actions. Cela évite de réécrire la logique d'accès dans chaque service.

Dans `product_service`, les droits d'écriture sont réservés aux vendeurs. Dans `order_service`, seuls les clients peuvent passer une commande. Cette règle est cohérente avec l'utilisation attendue de la plateforme.

```
def _token_claim(request, claim_name: str) -> str | None:
    user = request.user
    token = getattr(user, "token", None)
    if token is None:
        return None
    claim_value = token.get(claim_name)
    if isinstance(claim_value, str) and claim_value:
        return claim_value
    return None

def _user_role(request) -> str | None:
    token_role = _token_claim(request, "role")
    if token_role:
        return token_role
    role = getattr(request.user, "role", None)
    if isinstance(role, str) and role:
        return role
    return None

def _is_seller(request) -> bool:
    return _user_role(request) == "seller"

def _is_admin(request) -> bool:
    return _user_role(request) == "admin"

def _username(request) -> str:
    token_username = _token_claim(request, "username")
    if token_username:
        return token_username
    username = getattr(request.user, "username", "")
    if isinstance(username, str) and username:
        return username
    return ""
```

```
class SellerWritePermission(BasePermission):
    def has_permission(self, request, view):
        if request.method in SAFE_METHODS:
            return True
        return _is_seller(request) or _is_admin(request)

class ProductOwnerPermission(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in SAFE_METHODS:
            return True
        if _is_admin(request):
            return True
        return _is_seller(request) and bool(obj.seller_username) and obj.seller_username == _username(request)
```

8. Déploiement Docker

Le déploiement local repose sur Docker Compose. Chaque service a son image, son port, ses variables d'environnement et son volume. Le compose démarre aussi les bases PostgreSQL, Qdrant et Nginx. L'intérêt est de pouvoir lancer tout le système avec une seule commande.

Les noms réseau internes sont importants. Dans les conteneurs, localhost désigne le conteneur lui-même. Le compose utilise donc des alias réseau comme userservice, productservice, orderservice et searchservice. Les volumes gardent les données, les images et les vecteurs.

```
main_service:
  ports:
    - "8000:8000"
  environment:
    PRODUCT_SERVICE_URL: http://productservice:8002
    SEARCH_SERVICE_URL: http://searchservice:8004

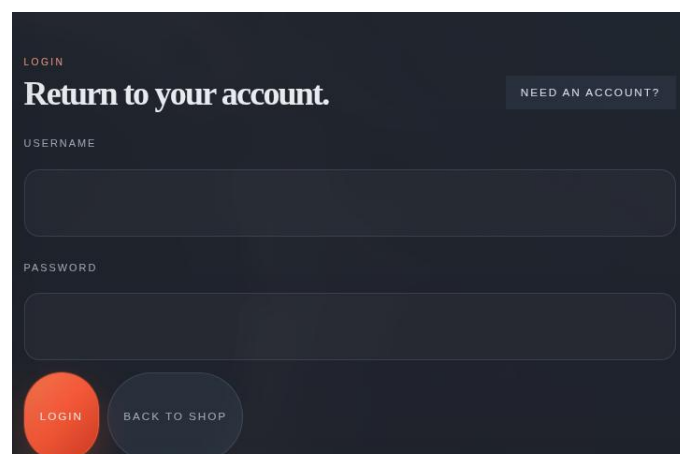
search_service:
  environment:
    QDRANT_URL: http://qdrant:6333
    DEFAULT_SEARCH_MODEL: dinov2_base_triplet_finetuned
```

Ce petit extrait résume l'idée du déploiement : des services indépendants, mais reliés par des URL internes stables.

9. Présentation des Interfaces

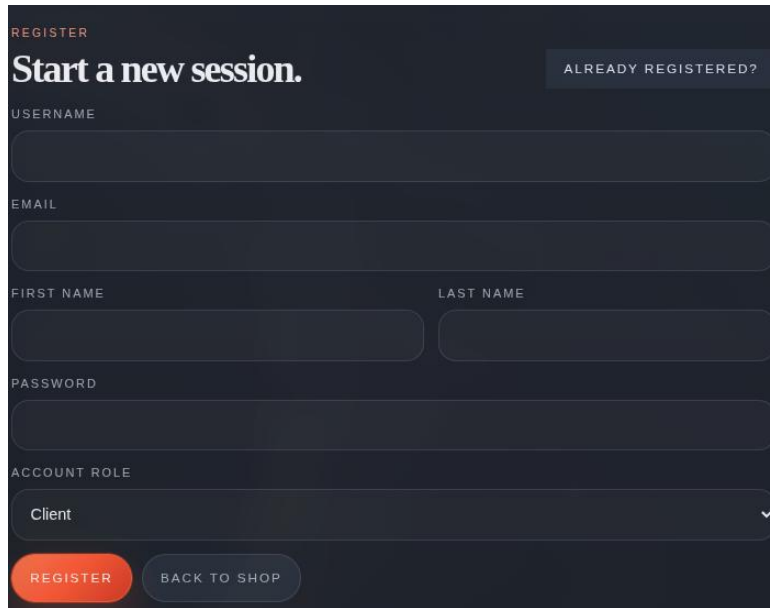
Nous présentons ici, à travers un enchaînement de captures d'écran, les scénarios d'exécution donnant un aperçu général du fonctionnement de notre application.

9.1. Page d'authentification



9.2. Formulaire d'inscription

La photo affiche le formulaire d'inscription.



REGISTER

Start a new session. [ALREADY REGISTERED?](#)

USERNAME

EMAIL

FIRST NAME LAST NAME

PASSWORD

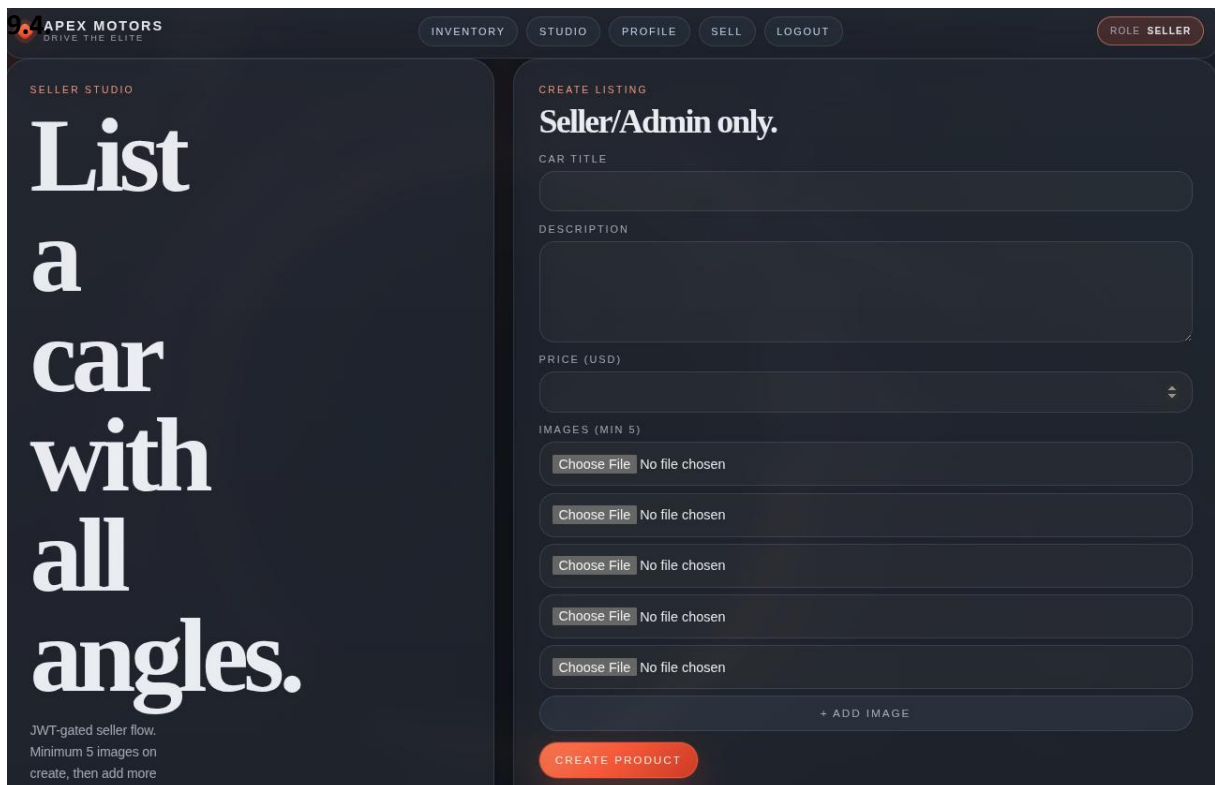
ACCOUNT ROLE

Client

REGISTER BACK TO SHOP

9.3. Page Vendeur

La photo illustre le formulaire que le vendeur doit remplir afin de préparer sa voiture pour la vente.



9.4 APEX MOTORS
DRIVE THE ELITE

INVENTORY STUDIO PROFILE SELL LOGOUT

ROLE SELLER

SELLER STUDIO

List a car with all angles.

JWT-gated seller flow.
Minimum 5 images on create, then add more images to the gallery.

CREATE LISTING

Seller/Admin only.

CAR TITLE

DESCRIPTION

PRICE (USD)

IMAGES (MIN 5)

Choose File No file chosen

Choose File No file chosen

Choose File No file chosen

Choose File No file chosen

Choose File No file chosen

+ ADD IMAGE

CREATE PRODUCT

9.5. Page Client

La photo affiche le résultat lorsque le client effectue une recherche.

7 products found
The response is already hydrated by main_service.

MATCHES

Top 10 hydrated product results ranked high to low. 7 RESULTS



CURATED SIMILARITY 95%

Clio RS

clio rs

\$4,499.98 SKU 199



CURATED SIMILARITY 95%

Clio faracha

clio faracha

\$4,000.00 SKU 197



KAGGLE CARS HYUNDAI ACCENT SEDAN 2012

SIMILARITY 82%

Hyundai Accent Sedan 2012

Car listing generated from the Kaggle Stanford car dataset class Hyundai Accent Sedan 2012.

\$57,249.00 SKU 130



KAGGLE CARS SCION XD HATCHBACK 2012

SIMILARITY 77%

Scion Xd Hatchback 2012

Car listing generated from the Kaggle Stanford car dataset class Scion Xd Hatchback 2012.

\$69,249.00 SKU 178



KAGGLE CARS NISSAN LEAF HATCHBACK 2012

SIMILARITY 76%

Nissan Leaf Hatchback 2012

Car listing generated from the Kaggle Stanford car dataset class Nissan Leaf Hatchback 2012.

\$67,249.00 SKU 170





La photo illustre le cas où le client sélectionne un véhicule parmi les résultats affichés et montre le formulaire pour effectuer l'achat.

DEDICATED DETAIL PAGE

Clio RS

Open the gallery, review the specs, and compare with similar cars below.

BUY NOW BACK TO INVENTORY CURRENT



clio rs

LISTING FACTS

Clio RS

\$4,499.98

YEAR	Current	MILEAGE	Ask seller
DRIVE	Listed	FUEL	Listed
GEARBOX	Automatic	CONDITION	Verified

BUY NOW

Move from interest to checkout.

NAME

EMAIL

BUY NOW

Ready when you are.



Conclusion

L'architecture d'Apex Motors illustre une mise en œuvre rigoureuse des concepts de systèmes distribués. En isolant les domaines métier au sein de microservices Django autonomes et en intégrant des technologies hétérogènes (PostgreSQL, Qdrant, Nginx), le projet atteint un équilibre entre robustesse transactionnelle et innovation IA. Cette structure, ancrée dans la séparation stricte de la persistance et l'orchestration centralisée via un gateway, constitue un socle technique performant et prêt pour une industrialisation future.